✅ Why Order of Execution is Important

Understanding SQL execution order helps you:

- Write correct queries
- Avoid errors
- Improve performance
- Understand how filtering and grouping actually work

⚠️ Important:
The order you WRITE SQL is NOT the order SQL EXECUTES it.

🔥 Actual SQL Execution Order

SQL processes queries in this logical order:

1. **FROM**
2. **JOIN**
3. **WHERE**
4. **GROUP BY**
5. **HAVING**
6. **SELECT**
7. **ORDER BY**
8. **LIMIT**

📌 Step-by-Step Explanation with Simple Meaning

1️⃣ FROM – Choose the Table

- SQL first decides **where the data is coming from**.
- It loads the table(s).

Example:

FROM messy_indian_dataset

👉 SQL picks the table first.

2️⃣ JOIN – Combine Tables (if used)

- If multiple tables are used, SQL joins them.

- It matches rows based on a condition.

Example:

JOIN states s ON m.state = s.state

👉 SQL combines matching rows from both tables.

3️⃣ WHERE – Filter Rows

- Removes unwanted rows.
- Works on **individual rows**.
- Cannot use aggregate functions here (like AVG, SUM).

Example:

WHERE purchase_amount > 1000

👉 Only rows greater than 1000 remain.

4️⃣ GROUP BY – Create Groups

- Groups rows based on a column.
- Required when using aggregate functions.

Example:

GROUP BY city

👉 All rows of same city are grouped together.

5️⃣ HAVING – Filter Groups

- Works after GROUP BY.
- Filters grouped data.
- Can use aggregate functions.

Example:

HAVING AVG(purchase_amount) > 1000

👉 Only cities with average > 1000 remain.

6️⃣ SELECT – Choose Columns

- Now SQL selects which columns to show.

- Calculates aggregate functions here.

Example:

SELECT city, AVG(purchase_amount)

👉 Final columns are chosen here.

⚠️ Important:
This happens **after** WHERE and GROUP BY.

7️⃣ ORDER BY – Sort Results

- Sorts final output.
- Works on final result.

Example:

ORDER BY purchase_amount DESC

👉 Highest values come first.

8️⃣ LIMIT – Restrict Rows

- Limits number of rows returned.

Example:

LIMIT 5

👉 Only first 5 rows shown.

📊 Example Breakdown

Query:

SELECT city, AVG(purchase_amount)

FROM messy_indian_dataset

WHERE purchase_amount > 1000

GROUP BY city

HAVING AVG(purchase_amount) > 2000

ORDER BY AVG(purchase_amount) DESC;

SQL Executes Like This:

1. FROM → Load table
2. WHERE → Keep rows > 1000

3. GROUP BY → Group by city
4. HAVING → Keep groups with avg > 2000
5. SELECT → Show city + avg
6. ORDER BY → Sort descending

🧠 **Simple Memory Trick**

Think of SQL like filtering water:

- FROM → Get water
- WHERE → Remove dirt
- GROUP BY → Separate into buckets
- HAVING → Remove bad buckets
- SELECT → Pour into glasses
- ORDER BY → Arrange glasses
- LIMIT → Serve only few

🚀 **Quick Comparison: WHERE vs HAVING**

WHERE	HAVING
Filters rows	Filters groups
Before GROUP BY	After GROUP BY
Cannot use AVG, SUM	Can use AVG, SUM

🎯 **Important Concept: Subquery Execution**

Example:

WHERE purchase_amount >

(SELECT AVG(purchase_amount) FROM messy_indian_dataset)

Execution:

1. Inner query runs first
2. Average is calculated
3. Outer query runs
4. Rows greater than average are selected

👉 Subquery runs first!

💡 Final Summary

Correct Execution Order:

FROM

JOIN

WHERE

GROUP BY

HAVING

SELECT

ORDER BY

LIMIT

🏆 Key Takeaways

✓ SQL does NOT execute from top to bottom
✓ WHERE cannot use aggregate functions
✓ HAVING is for grouped data
✓ SELECT runs near the end
✓ Subqueries run first